FastAPI：核心原理、代码实战

第一部分：FastAPI 基础知识

1. FastAPI 核心优势、与 Flask/Django 的区别

核心优势：

FastAPI 是一个**现代、快速（高性能）**的 Web 框架，基于 Python 3.7+ 的标准类型提示。

1. **极速性能**：基于 Starlette（Web 部分）和 Pydantic（数据校验），性能可与 NodeJS 和 Go 比肩（感谢 Uvicorn 这个 ASGI 服务器）。
2. **自动文档**：写完代码自动生成交互式 API 文档（Swagger UI / ReDoc），前端/测试人员直接在浏览器里调接口。
3. **类型提示 (Type Hints)**：利用 Python 的 `typing` 模块，不仅代码可读性强，IDE 还能给你超强的补全和纠错。
4. **异步原生支持**：天生支持 `async/await`，处理高并发 IO 轻而易举。
5. **数据校验强大**：集成 Pydantic，数据解析和校验非常严谨。

与 Flask/Django 的对比：

特性	FastAPI	Flask	Django
类型	现代异步框架	微框架 (WSGI)	全栈框架 (WSGI)
性能	极高 (ASGI + 异步)	中等	中等
数据校验	内置 Pydantic (强)	需手动/第三方库	自带 ORM 校验
异步支持	原生支持	基本不支持 (需用插件)	3.1+ 开始部分支持
API 文档	自动生成 (Swagger)	需手动安装插件	需手动安装插件
适用场景	现代 API、微服务、高并发	小型项目、灵活定制	大型全栈应用、Admin 后台

2. FastAPI 异步接口的实现、并发模型

原理：

FastAPI 是一个 **ASGI (Asynchronous Server Gateway Interface)** 框架。

- **并发模型**：基于协程 (Coroutine)。当一个请求在等待数据库查询或网络 IO 时 (`await` 时)，服务器可以去处理其他请求，极大提高了吞吐量。
- **服务器**：必须使用 ASGI 服务器运行，最常用的是 **Uvicorn**。

代码实战：

Code block

```
# main.py
from fastapi import FastAPI
import time
import asyncio

app = FastAPI()

# 1. 异步接口 (推荐用于 IO 密集型)
@app.get("/async-slow")
async def async_slow_endpoint():
    # 假设这是一个耗时 2 秒的数据库查询或网络请求
    # 注意要用 asyncio.sleep，不能用 time.sleep!
    await asyncio.sleep(2)
    return {"message": "异步任务完成"}

# 2. 同步接口 (兼容旧代码，用于 CPU 密集型)
@app.get("/sync-slow")
def sync_slow_endpoint():
    # 如果是普通的 def，FastAPI 会自动把它放到线程池中运行
    time.sleep(2)
    return {"message": "同步任务完成"}
```

运行方式：

Code block

```
pip install fastapi uvicorn
uvicorn main:app --reload
```

3. Pydantic 数据校验、模型定义的核心用法

Pydantic 的作用：

Pydantic 是 FastAPI 的灵魂。它负责**数据解析**和**数据校验**。

- 它会把前端传来的 JSON 自动转换成 Python 对象。
- 如果类型不对（比如把字符串传给了数字字段），它会自动返回清晰的错误信息。

代码实战：

Code block

```python
from fastapi import FastAPI
from pydantic import BaseModel, Field, EmailStr
from typing import Optional, List

app = FastAPI()

# 1. 定义 Pydantic 模型 (继承 BaseModel)
class UserCreate(BaseModel):
    # ... 表示必填字段
    username: str = Field(..., min_length=3, max_length=20, description="用户名")
    password: str = Field(..., min_length=6, description="密码")
    email: EmailStr  # 自动校验邮箱格式
    age: Optional[int] = Field(None, ge=0, lt=150)  # 可选，年龄 0-150
    tags: List[str] = []  # 字符串列表

# 2. 定义响应模型 (用于过滤返回给前端的字段，比如不返回密码)
class UserOut(BaseModel):
    username: str
    email: str
    age: Optional[int]

    class Config:
        orm_mode = True  # 允许直接读取 ORM 对象

# 3. 使用模型
@app.post("/users/", response_model=UserOut)
def create_user(user: UserCreate):
    # user 已经是解析并校验好的对象了
    print(f"收到用户: {user.username}")
    # 这里可以存入数据库...
    return user  # FastAPI 会自动根据 UserOut 过滤字段
```

4. FastAPI 依赖注入系统的原理与使用

原理：

依赖注入 (Dependency Injection) 是 FastAPI 最强大的功能之一。

- **目的：解耦**。把重复的逻辑（比如：获取当前用户、获取数据库连接、权限校验）抽离出来，让接口函数保持干净。
- **核心：** `Depends()` 。

代码实战：

Code block

```python
from fastapi import FastAPI, Depends, HTTPException, status

app = FastAPI()

# 1. 定义一个依赖项 (其实就是一个普通函数)
def common_parameters(q: str = None, skip: int = 0, limit: int = 100):
    return {"q": q, "skip": skip, "limit": limit}

# 2. 使用依赖项 (接口参数注入)
@app.get("/items/")
def read_items(commons: dict = Depends(common_parameters)):
    return commons

# 3. 进阶：依赖项类 + 权限校验
class FakeDB:
    def get_user(self, token: str):
        if token == "supersecret":
            return {"username": "admin", "role": "admin"}
        return None

db = FakeDB()

def get_current_user(token: str):
    user = db.get_user(token)
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid credentials"
        )
    return user

# 这个接口必须带正确的 token 才能访问
@app.get("/users/me")
def read_users_me(current_user: dict = Depends(get_current_user)):
    return {"user": current_user}
```

5. WebSocket 长连接的实现、异常处理

原理：

HTTP 是“请求-响应”模式，服务器不能主动推消息。WebSocket 是 TCP 长连接，**客户端和服务器可以双向实时通信**（比如聊天应用、实时看板）。

代码实战：

Code block

```python
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from typing import List

app = FastAPI()

# 简单的连接管理器
class ConnectionManager:
    def __init__(self):
        self.active_connections: List[WebSocket] = []

    async def connect(self, websocket: WebSocket):
        await websocket.accept()
        self.active_connections.append(websocket)

    def disconnect(self, websocket: WebSocket):
        self.active_connections.remove(websocket)

    async def broadcast(self, message: str):
        for connection in self.active_connections:
            await connection.send_text(message)

manager = ConnectionManager()

@app.websocket("/ws/{client_id}")
async def websocket_endpoint(websocket: WebSocket, client_id: str):
    await manager.connect(websocket)
    try:
        while True:
            # 接收客户端消息
            data = await websocket.receive_text()
            # 广播给所有人
            await manager.broadcast(f"Client {client_id} says: {data}")
    except WebSocketDisconnect:
        manager.disconnect(websocket)
        await manager.broadcast(f"Client {client_id} left the chat")
```

6. FastAPI 接口文档、异常处理、中间件开发

A. 接口文档

FastAPI 自动生成，无需任何配置：

- **Swagger UI** (交互式调试): 访问 `http://127.0.0.1:8000/docs`
- **ReDoc** (美观文档): 访问 `http://127.0.0.1:8000/redoc`

B. 异常处理

Code block

```python
from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import JSONResponse

app = FastAPI()

# 1. 基础 HTTP 异常
@app.get("/items/{item_id}")
def read_item(item_id: int):
    if item_id == 1:
        # 主动抛出异常
        raise HTTPException(status_code=404, detail="Item not found")
    return {"item_id": item_id}

# 2. 自定义异常
class MyCustomException(Exception):
    def __init__(self, name: str):
        self.name = name

# 3. 全局异常处理器
@app.exception_handler(MyCustomException)
async def custom_exception_handler(request: Request, exc: MyCustomException):
    return JSONResponse(
        status_code=418,
        content={"message": f"Oops! {exc.name} did something wrong."}
    )

@app.get("/custom-error/{name}")
def trigger_custom_error(name: str):
    if name == "test":
        raise MyCustomException(name=name)
    return {"name": name}
```

C. 中间件开发

中间件可以在请求到达接口**之前**和响应返回**之后**做一些操作（比如记录日志、CORS 处理、计时）。

Code block

```
import time
from fastapi import FastAPI, Request

app = FastAPI()

@app.middleware("http")
async def add_process_time_header(request: Request, call_next):
    # 1. 请求前：记录开始时间
    start_time = time.time()

    # 2. 处理请求
    response = await call_next(request)

    # 3. 响应后：计算耗时，添加到响应头
    process_time = time.time() - start_time
    response.headers["X-Process-Time"] = str(process_time)
    return response
```

第二部分：高频面试题解答

1. 为什么选择 FastAPI 而不是 Flask?

回答思路：从**开发效率**和**运行效率**两个维度答。

- **开发体验**：FastAPI 有 Pydantic 自动校验数据，不用写 `if not request.json.get('xxx')` 这种重复代码。而且自动生成文档，Flask 需要手动装 `flask-restx` 等插件。
- **类型安全**：基于 Python Type Hints，IDE 补全好，代码不容易出 BUG。
- **性能与并发**：Flask 是 WSGI 框架，同步阻塞。FastAPI 是 ASGI，原生支持异步，在处理高并发 IO（如调用第三方 API、查数据库）时，吞吐量比 Flask 高得多。
- **总结**：如果是写现代 API 接口，FastAPI 是趋势；Flask 更适合非常轻量的脚本化工具或旧项目维护。

2. FastAPI 的异步是怎么实现的?

回答思路：讲清楚 ASGI 和协程。

- **底层基石 (ASGI)**：FastAPI 基于 Starlette，这是一个 ASGI 框架。WSGI 一次只能处理一个请求，ASGI 支持异步。
- **服务器 (Uvicorn)**：FastAPI 必须跑在 Uvicorn (或 Hypercorn) 上。Uvicorn 是基于 uvloop 和 httptools 实现的，非常快。
- **调度机制**：当你在接口里写 `await db.query(...)` 时，Uvicorn 会挂起当前协程，去处理其他请求，等数据库返回结果了，再切回来继续执行。
- **注意**：如果在异步接口里写了同步阻塞代码（比如 `time.sleep()`），还是会堵住整个事件循环。

3. Pydantic 的作用是什么？

回答思路：核心是**解析**和**校验**。

- **数据解析 (Parsing)**：它能把前端传来的 JSON 字符串自动转换成 Python 的数据类型（比如字符串转 datetime，字符串转 int）。
- **数据校验 (Validation)**：如果前端传的数据不对（比如年龄传了负数，邮箱格式不对），Pydantic 会在请求到达你的业务代码之前就抛出异常，FastAPI 会自动返回格式非常友好的 422 错误。
- **模型复用**：可以定义 `UserCreate` (注册用) 和 `UserOut` (返回用)，配合 FastAPI 的 `response_model`，自动过滤掉密码等敏感字段。
- **ORM 兼容**：Pydantic 的 `orm_mode` 可以直接把 SQLAlchemy 的对象转换成 JSON 响应。

（注：文档部分内容可能由 AI 生成）